

Semantic GraphRAG

Knowledge Graph Pipeline over Interconnected XML Documents

[Cell-by-Cell Code Documentation](#) · [Cell 6](#) → [Cell 20](#)

RDF-based Knowledge Graph | NetworkX + Azure OpenAI | Databricks / PySpark
Offline graph construction · Online graph retrieval & answering

How to read this document

This system turns ~20,000 interconnected XWiki XML documents into a **semantic knowledge graph**, then answers questions by traversing that graph instead of doing flat vector search. Each document becomes a node; each hyperlink between documents becomes an edge whose label (predicate) is a real relationship such as `uses`, `part_of`, or `depends_on` — inferred by an LLM from the sentence around the link.

The pipeline has two phases. The **offline phase** (Cells 6–12) builds and visualizes the graph once. The **online phase** (Cells 18–20) answers each user query using the graph. Every cell below shows its role, its core logic, and a diagram of what it does to the data.

The one idea that makes this GraphRAG and not plain RAG: edges carry **meaning**. Cell 9 uses an LLM to turn each link's sentence into a specific predicate. The retriever (Cell 18) then **follows those meaningful edges** to gather context a flat search would miss.

Offline Phase — Build the Graph

Runs once. Input: cleaned XML documents. Output: a saved NetworkX knowledge graph with semantic edges.

Cell 6 — Knowledge Node Creation

Stage 4

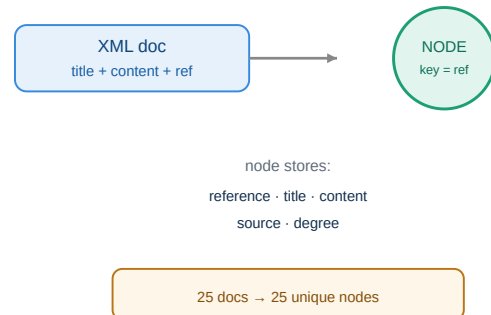
ROLE: ONE DOCUMENT → ONE NODE

Each XML document becomes exactly one graph node. The node is keyed by its **reference** (e.g. `Temp_Import.7a97...WebHome`) — a unique ID — while the title is kept only for display.

Core logic

- Select `reference`, `title`, `content` + metadata.
- Deterministic dedupe: keep the row with the most content (not a random one).
- Flag stub nodes (empty content) so the retriever won't treat them as answers.

Why reference, not title? Titles repeat and change; the reference uniquely identifies a document, so edges always point to the right node.



Cell 7 — Hyperlink Extraction

Stage 5

ROLE: FIND THE LINKS BETWEEN DOCUMENTS

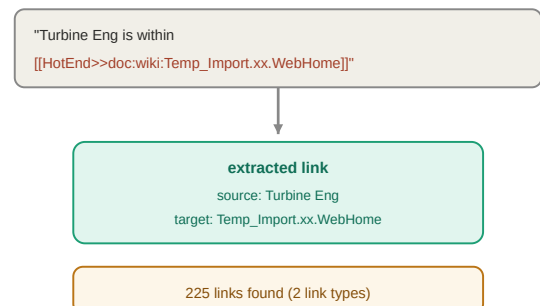
Scans each node's content for wiki hyperlinks and pulls out the target document reference. Two link types are recognized:

`doc:wiki:Temp_Import...` and `doc:Temp_Import...`

Core logic

- Regex finds every `[[ ... ]]` block.
- Split on `>>`: left = display text, right = target reference.
- Normalize the reference (drop `||anchor`, `#section`, whitespace).
- Runs distributed via a UDF so it scales to 20k docs.

Extraction only. This cell just lists links (source → target). Edges aren't built yet — that happens in Cell 11.



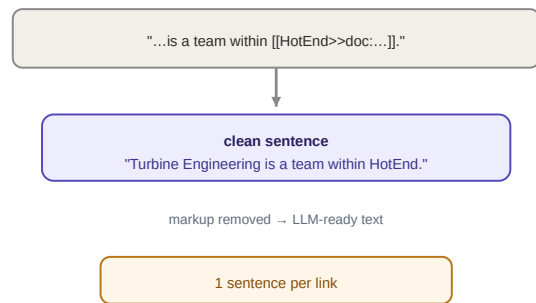
ROLE: FIND THE SENTENCE AROUND EACH LINK

For every extracted link, locate the sentence in the document that contains it, then strip the wiki markup to produce a clean, readable sentence for the LLM.

Core logic

- Split content into sentences (handles macros, headers, style tags).
- Find which sentence holds each link.
- Clean markup: `[[HotEnd>>doc:...]]` → `HotEnd` .
- Drop self-loops; runs distributed for scale.

Why this matters: the predicate quality in Cell 9 depends entirely on getting a clean, meaningful sentence here. Garbage sentence → garbage relationship.

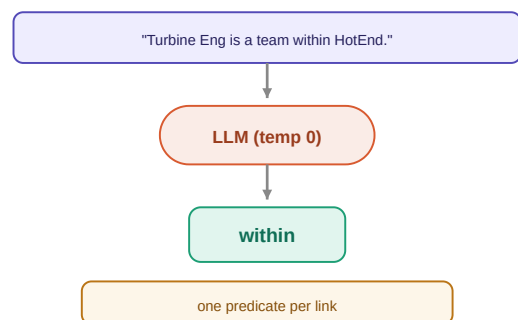
**Cell 9 — Semantic Predicate Extraction (LLM)****ROLE: TURN A SENTENCE INTO A RELATIONSHIP**

The heart of the pipeline. Each clean sentence goes to the LLM (temperature 0), which returns **one snake_case predicate** describing how the two documents relate — not a generic "linked to".

Core logic

- Strong system prompt + few-shot examples steer specific predicates.
- Threaded concurrency for speed; retry + backoff on rate limits.
- Output cleaned to pure snake_case; failures marked distinctly.

This is what makes or breaks the graph. Result: ~15 distinct predicates (`uses` , `part_of` , `owned_by` ...) with almost no `related_to` . That means the edges carry real meaning.



ROLE: PACKAGE RELATIONSHIPS AS TRIPLES

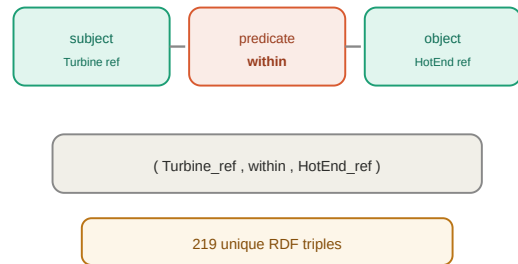
Combine source, predicate, and target into RDF triples

(**subject**, **predicate**, **object**), using **references** as subject and object so each triple uniquely identifies two documents.

Core logic

- Filter incomplete triples and self-loops.
- Drop **extraction_failed** (LLM errors aren't real edges).
- Deduplicate on (subject, predicate, object).
- Keep titles as display-only columns.

Result: 219 clean, unique triples — the formal representation of every relationship in the graph.



Cell 11 — Knowledge Graph Construction

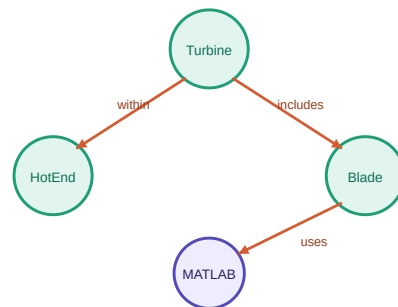
ROLE: ASSEMBLE THE ACTUAL GRAPH

Builds a NetworkX **MultiDiGraph**. All documents are added as nodes first (so isolated docs survive), then each triple becomes a directed edge carrying its predicate and source sentence.

Core logic

- Add all nodes with title + content + source.
- Add edges from triples: predicate + sentence stored on each edge.
- Compute and store **degree** on each node (for ranking later).
- MultiDiGraph keeps parallel edges (A can both **use** and **reference** B).

Result: 201 nodes, 219 edges. 25 loaded docs + 176 "shell" nodes (linked but not yet loaded — resolve at full 20k scale).



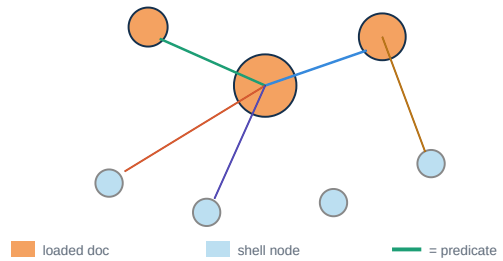
ROLE: SEE AND VERIFY THE GRAPH

Draws the graph so you can visually confirm nodes, edges, and predicates. Loaded docs are shown large with titles; shell nodes are small; each edge is colored by predicate type with a legend.

Core logic

- Matplotlib for a static image (spring layout, degree-sized nodes).
- Optional pyvis for an interactive, zoomable HTML graph.
- Edge labels show the predicate; hover shows the source sentence.

Practical note: at 201 nodes a full static image is dense — the readable "core" view (loaded docs only) is best for confirming relationships.



Online Phase — Answer a Query

Runs per user question. Input: a query + the saved graph. Output: an LLM answer grounded in retrieved documents.

Cell 16 — Query Understanding + Traversal + Ranking

Stage 12 · full retriever

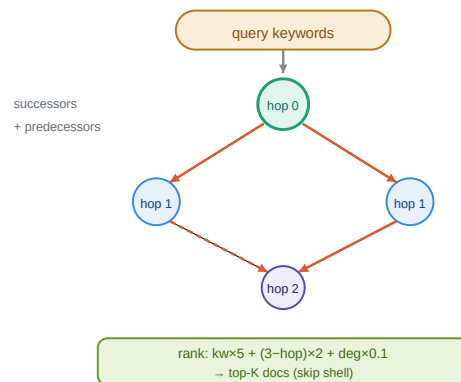
ROLE: THE COMPLETE RETRIEVAL ENGINE

The detailed retriever. It also collects `retrieved_edges` (the relationships walked) and tracks each node's **hop distance** from the start, with a fallback that expands to hop 3 when context is too sparse.

Core logic — the `explore()` function

- **Start nodes:** keyword score = $\text{title} \times 3 + \text{content} \times 1$, take top 3.
- **explore(frontier, hop):** for each node, visit `successors` & `predecessors`, record the edge (MultiDiGraph-safe: takes the first parallel edge), and tag new nodes with their hop number.
- **Fallback:** if fewer than 5 edges after hop 2, run hop 3.
- **Rank:** $\text{kw} \times 5 + (3 - \text{hop}) \times 2 + \text{degree} \times 0.1$, skip shell nodes, return top-K.

Why track hops and edges? Hop distance drives ranking (closer = more relevant), and `retrieved_edges` lets the context builder show the LLM the actual relationships, not just the documents.



Cell 18 — Top-K Graph Retriever (No LLM)

Stage 12 · simplified retriever

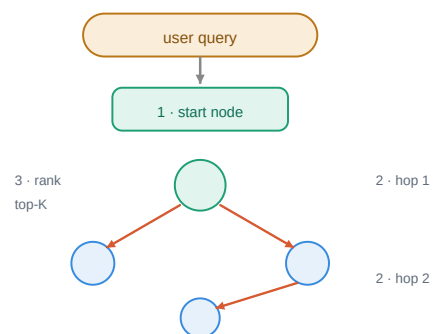
ROLE: FIND THE RIGHT DOCUMENTS VIA THE GRAPH

The streamlined, output-focused version of Cell 16. Same 3-step logic (start node → traverse → rank), but wrapped in a clean `retrieve_candidates()` function that returns the top-K documents with **title, reference, and content** for display and for feeding the LLM.

Core logic — three steps

1. **Find start nodes:** keyword overlap between query and each node's title + content.
2. **Traverse:** walk 1–2 hops out along edges (both directions) to collect connected docs.
3. **Rank:** score by keyword relevance + hop closeness + degree; return top-K.

The GraphRAG difference: flat RAG returns only the doc the query names. Traversal pulls in neighbors along real relationships — so multi-hop questions get answered.



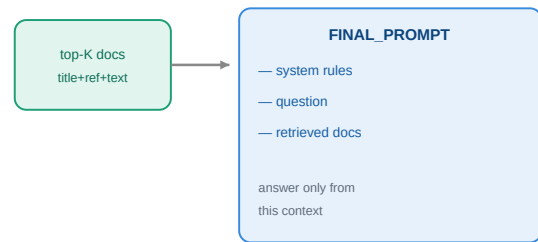
ROLE: TURN RETRIEVED DOCS INTO A PROMPT

Takes the top-K candidates and assembles the prompt the LLM will see: a system prompt with strict rules, the user's question, and the retrieved document text (capped per doc to protect the context window).

Core logic

- Format each candidate: title + reference + capped content.
- System prompt: answer **only** from context, be analytical, cite titles.
- Assemble `FINAL_PROMPT` = question + retrieved documents.

Key ordering: the prompt is built **after** retrieval. The LLM never sees the graph until context is gathered — the correct GraphRAG sequence.



Cells 19–20 — LLM Answer Generation

Stage 12

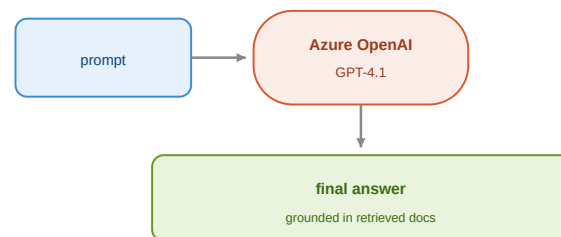
ROLE: GENERATE THE GROUNDED ANSWER

Cell 19 sets up the Azure OpenAI client. Cell 20 sends the system prompt + final prompt and prints the answer. The model is instructed to answer only from the retrieved context and say so if the information isn't present.

Core logic

- `llm.invoke([system, human])` with the built prompt.
- Guard: skip the call if no docs were retrieved.
- Error handling around the API call.

Grounded, not hallucinated: because the LLM only sees retrieved graph context, answers stay tied to the actual documents — with the relationships traversal surfaced.



Pipeline Summary

Offline (build once): clean XML → one node per doc (Cell 6) → extract links (7) → find link sentences (8) → LLM infers predicate (9) → RDF triples (10) → build graph (11) → visualize (12).

Online (per query): query → find start node + traverse + rank top-K (Cell 18) → build prompt from retrieved docs → LLM answers only from that context (19–20).

Why it is real GraphRAG: nodes are keyed by unique reference; edges carry LLM-inferred meaning (not just "linked"); retrieval traverses those meaningful edges to gather multi-hop context; and the prompt is built only after retrieval. The single most important cell is **Cell 9** — it is what turns a plain hyperlink graph into a semantic knowledge graph.